

MEDIATEK

MT8676 Hypervisor Display User Manual

Version: 1.1
Release date: 2024-11-13

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-10-28	Scott Wang	Official release
1.1	2024-11-13	Scott Wang	- Added Tbox in Section 1.2.2.2 Hypervisor (Yocto + Android + Tbox) - Added Table 1-3. Connector device node introduction

Table of Contents

Version History	2
Table of Contents	3
List of Figures	3
List of Tables	3
1 Display & Multi-Screen	4
1.1 Overview	4
1.1.1 Brief Introduction	4
1.1.2 Abbreviation	4
1.1.3 Capability	4
1.2 Architecture/Process Overview	5
1.2.1 HW Architecture	5
1.2.2 SW Architecture	6
1.3 Configuration/Customization Guideline	8
1.3.1 Host Yocto Configuration	8
1.3.2 Guest Android Configuration	9
1.4 Frequently Asked Questions/Troubleshooting	10
1.4.1 hwc CMD	10
1.4.2 Display Driver CMD	10
1.4.3 Pattern	11
1.4.4 Black Screen	12
1.4.5 Screen Distortion/Flickering	12
1.4.6 Screen Freeze/Lag	12
Exhibit 1 Terms and Conditions	14

List of Figures

Figure 1-1. Display HW architecture	5
Figure 1-2. Display Android architecture	6
Figure 1-3. Display Yocto + Android architecture	7

List of Tables

Table 1-1. Abbreviation	4
Table 1-2. Capability	4
Table 1-3. Connector device node introduction	8

1 Display & Multi-Screen

1.1 Overview

1.1.1 Brief Introduction

This chapter mainly introduces the MT8676 display architecture and multi-screen configuration guide.

1.1.2 Abbreviation

Table 1-1. Abbreviation

Abbreviation	Explanation
CRTC	Cathode Ray Tube Controller
DP	DisplayPort
DPI	Digital Parallel Interface
DSC	Display Stream Compression
DSI	Display Serial Interface
DTS	Device Tree
HWC	Hardware Composer
LCM	Liquid Crystal Display Module
SerDes	Serializer/Deserializer

1.1.3 Capability

Table 1-2. Capability

Name	Capability
DISP pipeline	10bit pipe x 3 (MAX 688MHz at 0.75V)
DSI	DSI0 + DSI1 C/D PHY Combo 4-lane x 2 DPHY: 2.5Gbps/lane
DP	DP1.4, 4-lane 8.1Gbps/lane (4-lane mode conflict with USB3)
Panel number	1 ~ 6
OVL layers	OVL0~OVL7: Total 16 layers
OVL De-comp.	AFBC (RGBA8888/RGB888/RGB565)
Compression	VESA DSC 1.2 (2 slice x 2)

1.2 Architecture/Process Overview

1.2.1 HW Architecture

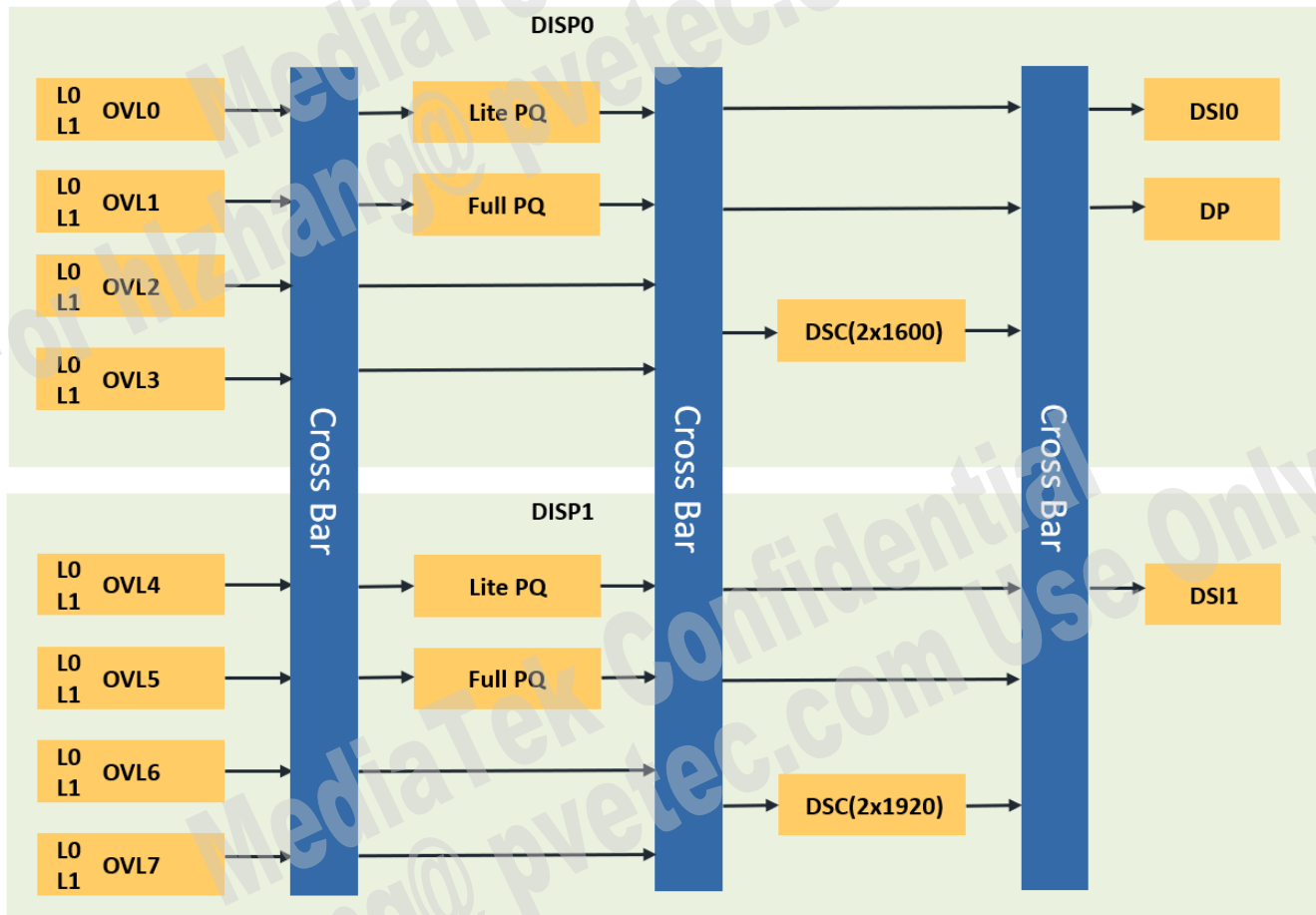


Figure 1-1. Display HW architecture

1.2.2 SW Architecture

1.2.2.1 Android

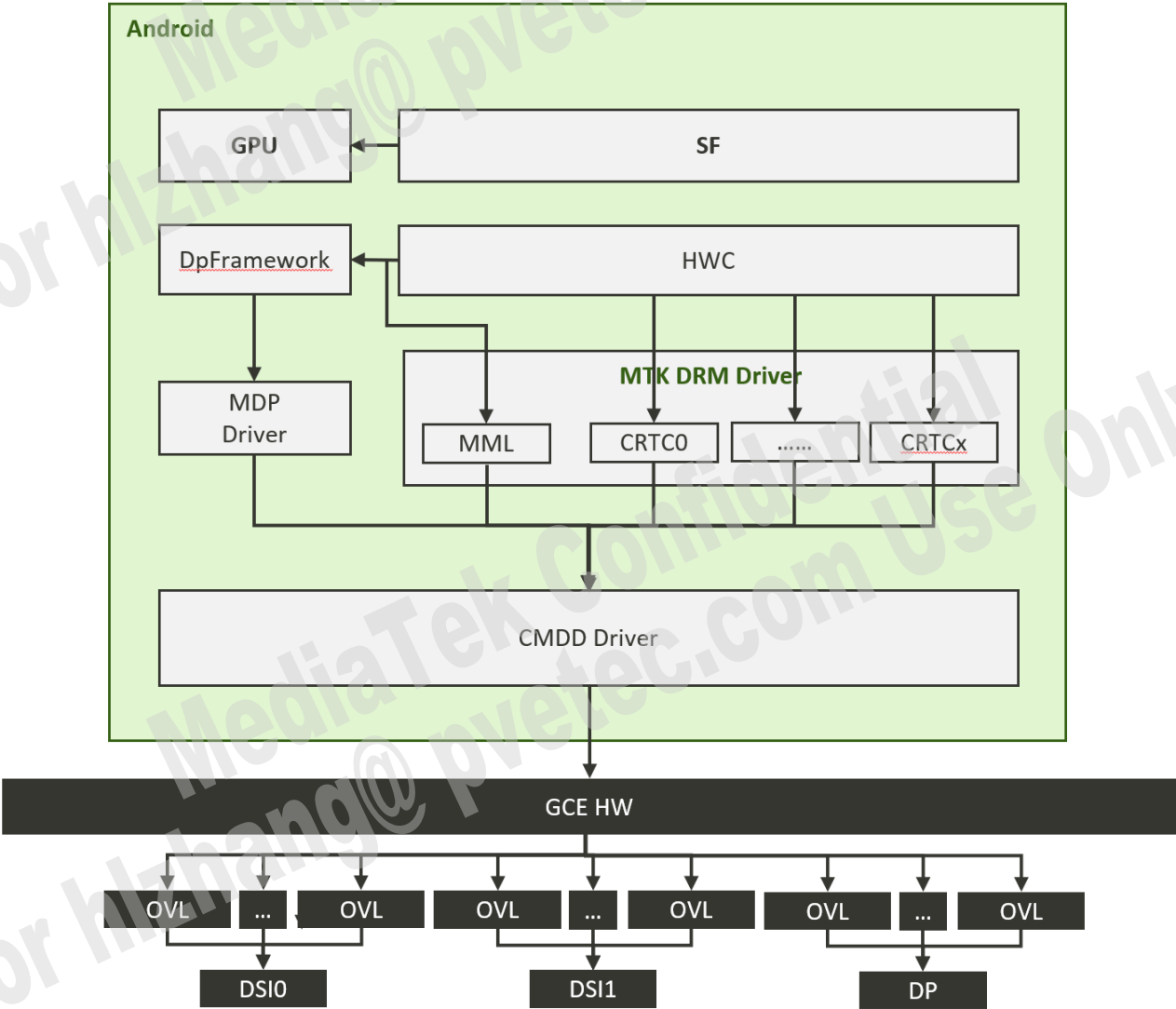


Figure 1-2. Display Android architecture

1.2.2.2 Hypervisor (Yocto + Android + Tbox)

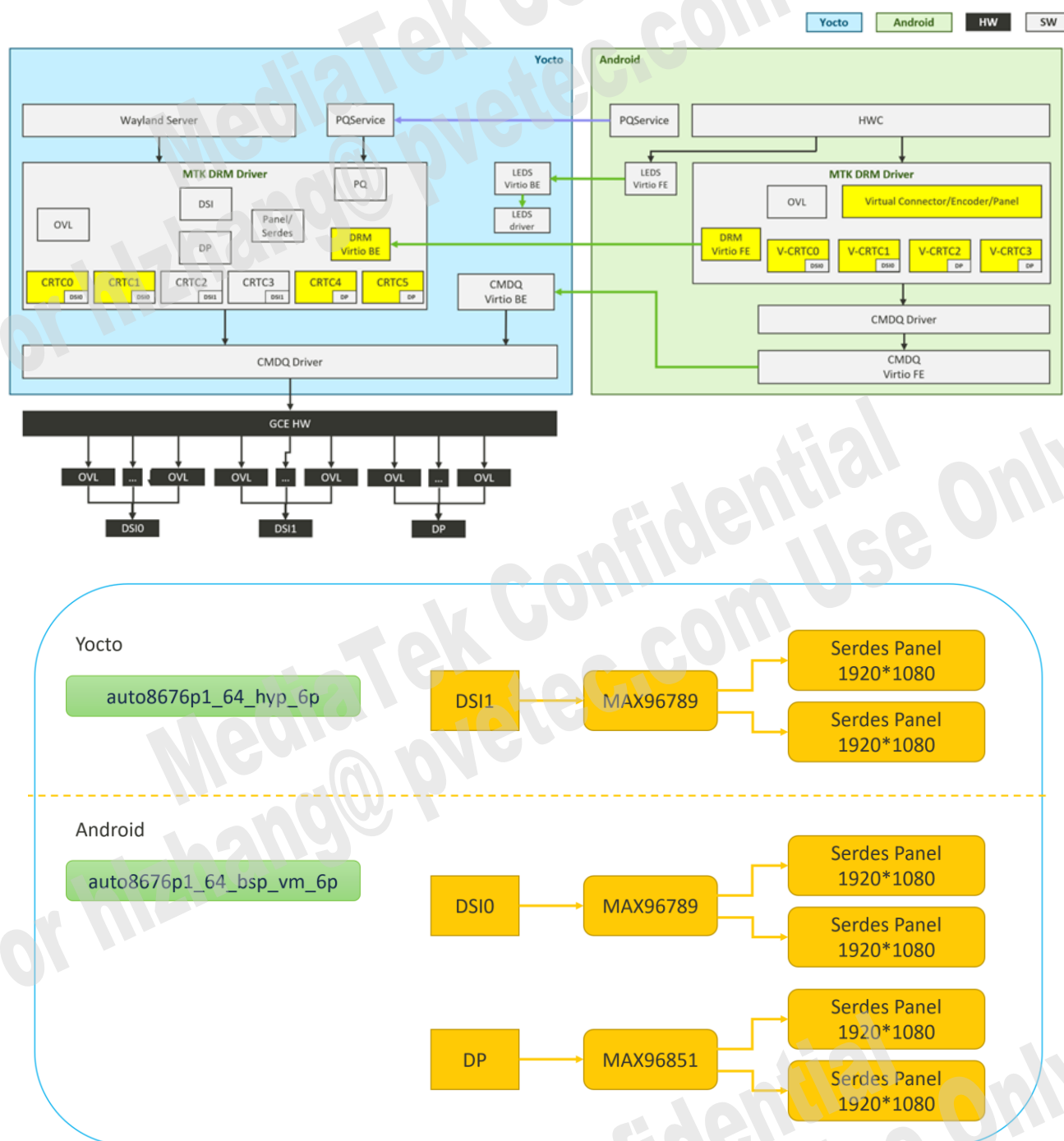


Figure 1-3. Display Yocto + Android architecture

A driver-level virtualization solution is currently being utilized. Disp_pq and disp interface (DSI/DP) are virtualized, while OVL remains in direct pass-through control, balancing performance considerations.

The complete CRTC driver (including OVL, disp_pq, dsi/dp) operates in Host Operating System. The Guest Operating System utilizes Virt-Connector/Encoder to access the Host Operating System's DRM driver via virtio to obtain display mode and control CRTC. The Guest OS updates layers using a pass-through method to directly drive OVL without the need for communication with the Host.

Taking MediaTek public project “auto8676p1_64_hyp_6p (yocto 2 screens + android 4 screens)” as an example. There are six CRTCs in Yocto Kernel, CRTC0/1(DSI0) and CRTC4/5(DP) are for Android screen, CRTC 2/3(DSI1) is for Yocto screen. In Android Kernel, connector/encoder is virtual, dts nodes are virt_dsi0_0/virt_dsi0_1/virt_dp_0/virt_dp_1.

Table 1-3. Connector device node introduction

Device Node	Introduction
dsi0 dsi1 dp_intf	Connector device node on host OS
virt_dsi0_0 virt_dsi1_0 virt_dp_0	Connector device node on guest OS
virt_dsi0_1 virt_dsi1_1 virt_dp_1	When enable superframe, connector device node for 2nd panel

1.3 Configuration/Customization Guideline

1.3.1 Host Yocto Configuration

The Kernel configuration of all multi-screen display (Android + Yocto) should be applied to Host Yocto Kernel, referred to the related chapter in *MT8676_Android_Display_User_Manual_V1.0*.

- Kernel config path:

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/

- Kernel dts path:

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/

- Kernel ko table path:

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/ko_order_table

You can refer to MediaTek public project: auto8676p1_64_hyp_6p (yocto 2 screens + android 4 screens), which is shown as below.

- Kernel config file:

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/auto8676p1_64_hyp_6p.defconfig

- Kernel dts file:

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/auto8676p1_64_hyp_6p.dts
meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/dtsi/cust_mt8676_display_config_hyp_6p.dtsi
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/cust_mt8676_display_interface.dtsi

Please note the following two points:

1. In the DP Serdes node, the width and height of the screen need to be filled into the crop-size.

```

&i2c11 {
    status = "okay";
    max96851: max96851@40 {
        status = "disabled";
        compatible = "maxim,max96851-dp";
        reg = <0x40>;
        pinctrl-names = "default";
        reset-gpios = <&pio 220 0>;

        panel-name = "BoTai";
        ser-super-frame = <0>;
        ser-dual-link = <0>;
        ser-dp-mst = <0>;
        support-touch = <1>;

        crop-size = <2560 1440 2560 1440>;

        feature{
            feature-handle-num = <1>;
            feature-handle0{
                check-cmd = <0x01 0x01 0x1a 0xb (
                verify-cmd = <0x01 0x00 0x1a 0x0(
                feature-setting = "HuaYang";
            };
        };
    };
}

```

- virt_XXX_1 node in DTS represents the second screen output of the interface. It is only necessary to enable it when superframe is enabled.

1.3.2 Guest Android Configuration

You can refer to MediaTek public project: auto8676p1_64_vm_6p.

Kernel dts file:

src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/cust_mt8676_display_config_vm_6p.dtsi

- The core change is to disable the original nodes representing the actual Hardware dsi/dp in dts and enable the virt_dsi/dp nodes. virt_XXX_1 node representing the second screen output of this interface, and only needs to be enabled when superframe is enabled.

```

&dsi0 {
    status = "disabled";
};

&mipi_tx_config0 {
    status = "disabled";
};

&virt_dsi0_0 {
    status = "okay";
};

&virt_dsi0_1 {
    status = "okay";
    dsi-supply = <&virt_dsi0_0>;
};

&virt_dp_0 {
    status = "okay";
};

&virt_dp_1 {
    status = "okay";
    dp-supply = <&virt_dp_0>;
};

```

- Add the CRTC OVL configuration. In general, the CRTC OVL configuration of the Guest OS should be consistent with the corresponding CRTC OVL configuration on the Host OS.

```
&dispsys_config {
    crtc0-ovl-path = <DDP_COMPONENT_OVL0_2L>,
    <DDP_COMPONENT_OVL1_2L>,
    <DDP_COMPONENT_OVL2_2L>,
    <DDP_COMPONENT_OVL3_2L>,
    <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;
    crtc-layer-nr = <8>;
    pinctrl-names = "mode_te_te";
    pinctrl-0 = <&mtkfb_pins_lcm_dsi_te>;
    status = "okay";
};
```

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 hwc CMD

- Log CMD:

```
adb shell setprop persist.vendor.debug.hwc.log V && adb shell setprop
vendor.debug.hwc.skip_log 0 && adb shell dumpsys SurfaceFlinger
```

- Dump sf info:

```
adb shell dumpsys SurfaceFlinger > sf.log
```

- Force GPU (GPU compose):

```
adb shell service call SurfaceFlinger 1008 i32 1
```

1.4.2 Display Driver CMD

“On” means enable log output, “Off” means disable log output.

Frequently used display mobile log:

```
adb shell "echo mobile:on > /d/mtkfb"
```

If you want to capture the boot log, you need to modify the code directly:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
bool g_mobile_log = 1;
```

More detailed log:

```
adb shell "echo detail:on > /d/mtkfb"
```

If you want to capture the boot log, you need to modify the code directly:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
bool g_detail_log = 1;
```

Fence log:

```
adb shell "echo fence:on > /d/mtkfb"
```

If you want to capture the boot log, you need to modify the code directly:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_fence_log = 1;
```

Irq log:

```
adb shell "echo irq:on > /d/mtkfb"
```

If you want to capture the boot log, you need to modify the code directly:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_irq_log = 1;
```

Get display diagnose dump log:

```
adb shell "echo diagnose > /sys/kernel/debug/mtkfb && cat /sys/kernel/debug/mtkfb" > mtkfb.txt
```

1.4.3 Pattern

First, you need to enable register debug.

```
diff --git a/drivers/clk/mediatek/clkbdg.c b/drivers/clk/mediatek/clkbdg.c
index 124dff7d05c1..960b294e6cf5 100644
--- a/drivers/clk/mediatek/clkbdg.c
+++ b/drivers/clk/mediatek/clkbdg.c
@@ -980,7 +980,7 @@ static int clkbdg_set_rate(struct seq_file *s, void *v)
     return r;
 }

-#if defined(CONFIG_MTK_ENG_BUILD)
+#if 1//defined(CONFIG_MTK_ENG_BUILD)
 static void *reg_from_str(const char *str)
 {
     static phys_addr_t phys;
@@ -2164,7 +2164,7 @@ static const struct cmd fn common_cmds[] = {
     CMDFN("disable_unprepare_provider", clkbdg_disable_unprepare_provider),
     CMDFN("set_parent", clkbdg_set_parent),
     CMDFN("set_rate", clkbdg_set_rate),
-#if defined(CONFIG_MTK_ENG_BUILD)
+#if 1//defined(CONFIG_MTK_ENG_BUILD)
     CMDFN("reg_read", clkbdg_reg_read),
     CMDFN("reg_write", clkbdg_reg_write),
     CMDFN("reg_set", clkbdg_reg_set),
```

- dsi0 pattern:

```
adb shell "echo reg_write 0x1400d178 0xc61 > /proc/clkbdg;cat /proc/clkbdg"
```

- dsi1 pattern:

```
adb shell "echo reg_write 0x1420d178 0xc61 > /proc/clkbdg;cat /proc/clkbdg"
```

- dp pattern:

```
adb shell "echo reg_write 0x1400bf00 0x41 > /proc/clkgdb;cat /proc/clkgdb"
```

1.4.4 Black Screen

- Is the backlight on
- If exist bridge ic, is the bridge ic OK?
- Is dsi/dp pattern OK
- Is screencap OK

```
adb shell screencap -d 0/1 /sdcard/1.png
```

After confirming that all the above check points are satisfactory, the display owner should make analysis.

1.4.5 Screen Distortion/Flickering

- Check the log to see if there is DISP_OVL/RDMA underflow/abnormal

- Check size setting

- cmd:

```
adb shell "echo mobile:on > /d/mtkfb"
```

- Check clk/dram frequency

- clk

- dump:

```
adb shell " echo fmeter > /proc/clkgdb ; cat /proc/clkgdb | grep -e disp"
```

- force max:

```
adb shell "echo 0 0 > /sys/module/mtk_mmdvfs_debug/parameters/force_step"
```

- dvfs

- dump:

```
adb shell "cat /sys/kernel/helio-dvfsrc/dvfsrc_dump | grep -e uv -e Mbps"
```

- force max:

```
adb shell "echo 0 > /sys/kernel/helio-dvfsrc/dvfsrc_force_vcore_dvfs_opp"
```

- Is force GPU OK

```
adb shell service call SurfaceFlinger 1008 i32 1
```

- Is dsi/dp pattern OK

1.4.6 Screen Freeze/Lag

- Check main log to see if there is fence timeout

```
GED : Frame didn't finished in 1000 ms (try = 1)
GED : Fence fd = 82
GED : fence SurfaceView - #0:2 0
GED : pt timeline-primary-0-13 sw_sync status(0) sync_drv(250117) 0.000000
GED : pt timeline-unknown-0-13 sw_sync status(0) sync_drv(250048) 0.000000
GED : pt timeline-external-3-16 sw_sync status(1) sync_drv(238184) 8203.513127
```

- Check kernel log to see if there is underflow/abnormal
- Enable fence log, check fence release

```
adb shell "echo fence:on > /d/mtkfb"
```

- User systrace to analysis performance issue

```
adb shell perfetto -o /data/misc/perfetto-traces/trace -t 10s sched freq idle am wm gfx  
view input
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.